

Florence: Architecture - Data Service

Overview

The Florence Data Service is a RESTful application programming interface built with FastAPI and Python 3.13. It functions as the primary backend for the Florence platform. The system interfaces directly with a Supabase database to manage user authentication, health data tracking, clinical workflows and global administration. The service handles data validation, business logic enforcement and secure communication with frontend clients.

Architecture

The application routes are divided into domain-specific modules. It implements a singleton proxy pattern for the database client to enable lazy initialisation. This design prevents missing environment variables from causing import errors during test execution. The service configures cross-origin resource sharing middleware to allow requests from the frontend application. It relies on JSON Web Tokens for secure endpoint access.

Environment Configuration

The system requires specific environment variables to function correctly. The developer must supply these values in a local configuration file or via the hosting platform settings.

- `SUPABASE_URL` : The specific project URL provided by the database host.
- `SUPABASE_SERVICE_KEY` : The master service role key required for administrative actions.
- `LLM_ENGINE_SERVICE_URL` : The optional endpoint for the external language model service used to generate synthetic patient data.

Directory Structure

The data service repository contains several core files and directories that govern its operation.

- `main.py` : The primary entry point that initialises the FastAPI application and configures the middleware.
 - `client.py` : The database configuration file that implements the singleton proxy pattern for the Supabase client.
 - `pyproject.toml` : The configuration file that defines project metadata and lists the necessary Python dependencies.
 - `vercel.json` : The deployment configuration file that instructs the hosting platform on how to route incoming requests.
 - `routers/` : A subdirectory containing all domain-specific application logic separated into modular files (`authentication.py` , `patients.py` , `clinicians.py` , `admin.py` and `chat_history.py`).
-

Package Management with uv

The project relies on a standard Python configuration file but the architecture is optimised for the `uv` package manager. The `uv` tool is an extremely fast Python package installer and resolver written in Rust. It significantly reduces the time required to install dependencies and manage virtual environments compared to traditional tools.

The developer can utilise `uv` to synchronise the project environment directly from the `pyproject.toml` file. This ensures all required packages such as FastAPI, Uvicorn and the Supabase client are installed rapidly and consistently across different development machines.

Installation Instructions

The developer must follow these steps to set up the local environment.

1. Install the `uv` package manager on the host system.
2. Execute the appropriate `uv` command to create a virtual environment and install the dependencies listed in the configuration file.
3. Populate the local environment file in the project root with the required database credentials.

4. Start the application using the Uvicorn server gateway interface.

API Modules

General

- `GET /` : Root endpoint for health checks and service verification.

Authentication Module

The authentication module processes account creation, login requests and session validation. The system assigns specific roles during registration. Supported roles include patients, clinicians, hospital administrators and global administrators.

- `POST /auth/register` : Creates a new user account. The system automatically provisions a database profile, creates default health thresholds and sets measurement unit preferences for new patients.
- `POST /auth/register_admin` : Allows an existing administrator to create new administrative accounts.
- `POST /auth/login` : Authenticates a user with an email and password to return a session object.
- `GET /auth/me` : Decodes the provided token to return the active user profile and role metadata.

Patient Module

The patient module provides self-service endpoints. The system implements an automatic unit conversion layer to translate measurements between the base metric format and the preferred display format for glucose and cholesterol.

Profile and Settings

- `GET /patients/me` : Retrieves the complete profile of the authenticated patient. The system signs the storage uniform resource locator for profile pictures automatically.
- `PUT /patients/me` : Updates editable profile fields. If the patient updates their height or weight, the system automatically calculates and records a new body mass index value.
- `GET /patients/me/settings` : Retrieves measurement unit preferences.
- `PATCH /patients/me/settings` : Modifies measurement unit preferences.
- `PATCH /patients/me/risk` : Updates the risk level and accompanying rationale generated by the artificial intelligence service.

Health Tracking

- `GET /patients/me/monitor-data` : Retrieves historical biometric measurements with optional pagination.
- `POST /patients/me/monitor-data` : Records new biometric measurements.
- `PUT /patients/me/monitor-data/{data_id}` : Modifies an existing biometric measurement entry.
- `GET /patients/me/daily-logs` : Retrieves dietary records and mealtime glucose readings.
- `POST /patients/me/daily-logs` : Creates or updates a daily dietary log.
- `GET /patients/me/activity-logs` : Retrieves physical activity records.
- `POST /patients/me/activity-logs` : Records physical activity details including duration and calories burned.
- `GET /patients/me/disease-logs` : Retrieves the active and resolved medical conditions.
- `POST /patients/me/disease-logs` : Records a new medical condition.
- `PATCH /patients/me/disease-logs/{log_id}` : Updates an existing medical condition log.
- `DELETE /patients/me/disease-logs/{log_id}` : Deletes a specific medical condition log.

Medications

- `GET /patients/me/medications` : Retrieves active medication profiles.
- `POST /patients/me/medications` : Adds a new medication profile.
- `PATCH /patients/me/medications/{med_id}` : Updates an existing medication profile.
- `DELETE /patients/me/medications/{med_id}` : Removes a medication profile.
- `GET /patients/me/medication-schedule` : Compiles medications and recent intake logs to display a comprehensive adherence schedule.
- `POST /patients/me/medication-intake` : Records an intake event.
- `DELETE /patients/me/medication-intake/{patient_medication_id}` : Removes an intake event logged on the current day.
- `GET /patients/medications/dictionary` : Retrieves the global medication dictionary for autocomplete.
- `GET /patients/medications/frequencies` : Retrieves available dosage frequencies.

Thresholds and Recommendations

- `GET /patients/me/thresholds` : Retrieves the defined minimum and maximum limits for biometric data points.
- `PUT /patients/me/thresholds` : Updates the biometric limits.
- `GET /patients/me/recommendations` : Retrieves active health suggestions.
- `POST /patients/me/recommendations` : Saves a new batch of health suggestions generated by the external artificial intelligence pipeline.

- PATCH /patients/me/recommendations/{rec_id} : Updates the status of a specific recommendation (e.g. completed or dismissed).

File Management and Device Actions

- POST /patients/me/avatar : Uploads a profile picture to object storage.
- POST /patients/me/clinical-documents : Creates a clinical document record directly via JSON.
- POST /patients/me/clinical-documents/upload : Uploads a medical document to object storage and creates a database reference.
- POST /patients/me/meal-photo : Uploads an image of a meal to object storage.
- POST /patients/me/automated-actions : Logs an automated action triggered by the patient's device.

Clinician Module

The clinician module enables medical personnel to oversee patient progress, adjust treatment plans and monitor critical alerts.

Clinician Management

- GET /clinicians/me : Retrieves the active clinician profile.
- PUT /clinicians/me : Modifies the clinician profile details.
- GET /clinicians/me/settings : Retrieves measurement unit preferences for the clinician display.
- PUT /clinicians/me/settings : Updates measurement unit preferences.
- GET /clinicians/medications/dictionary : Retrieves the global medication dictionary for clinicians.
- GET /clinicians/medications/frequencies : Retrieves the global dosage frequencies for clinicians.

Patient Oversight

- GET /clinicians/me/patients : Retrieves a list of all patients currently assigned to the clinician.
- GET /clinicians/available-patients : Retrieves a list of unassigned patients.
- POST /clinicians/patients/{patient_id}/assign : Assigns a specific patient to the authenticated clinician.
- POST /clinicians/patients/{patient_id}/unassign : Removes a specific patient from the clinician caseload.
- GET /clinicians/me/patients/{patient_id} : Retrieves comprehensive records for a specific assigned patient.

- PUT /clinicians/me/patients/{patient_id}/profile : Updates the demographic profile of an assigned patient.
- PUT /clinicians/me/patients/{patient_id}/assess-risk : Overrides the calculated risk tier for an assigned patient.
- POST /clinicians/me/patients/{patient_id}/notes : Attaches a clinical observation note to an assigned patient.

Alerts and Data Management

- GET /clinicians/me/alerts : Generates a list of priority alerts by evaluating recent patient biometric data against established thresholds.
- GET /clinicians/me/patients/{patient_id}/thresholds : Retrieves thresholds for an assigned patient.
- PUT /clinicians/me/patients/{patient_id}/thresholds : Modifies thresholds for an assigned patient.
- POST /clinicians/patients/{patient_id}/medications : Prescribes a new medication.
- PUT /clinicians/medications/{medication_id} : Updates dose, type form layout definitions or frequency schedules for a medication.
- DELETE /clinicians/medications/{medication_id} : Discards a medication entry permanently.
- POST /clinicians/patients/{patient_id}/diseases : Diagnoses a new medical condition.
- GET /clinicians/patients/{patient_id}/diseases : Fetches the full medical condition logs containing dates, status and IDs for a patient.
- PUT /clinicians/diseases/{disease_id} : Updates an existing condition record.
- DELETE /clinicians/diseases/{disease_id} : Permanently purges a medical condition record.

Administrator Module

The administrator module provides system oversight, data management and synthetic data generation tools. Only authorised global or hospital administrators may access these paths.

System Management

- GET /admin/patients : Compiles a list of all patients alongside their latest alert status.
- PUT /admin/patients/{patient_id} : Edits any patient profile.
- DELETE /admin/patients/{patient_id} : Completely removes a patient account from the database and the authentication provider.
- DELETE /admin/patients/{patient_id}/data : Erases all recorded health logs while preserving the user account.

- `PUT /admin/patients/{patient_id}/assign-clinician` : Manually pairs a patient with a clinician.
- `GET /admin/clinicians` : Retrieves all clinician profiles and their associated patient counts.
- `PUT /admin/clinicians/{clinician_id}` : Modifies a clinician profile.
- `DELETE /admin/clinicians/{clinician_id}` : Removes a clinician profile and their associated authentication account.
- `GET /admin/organisations` : Retrieves all registered organisations.
- `POST /admin/organisations` : Creates a new organisation entry.
- `PATCH /admin/organisations/{org_id}` : Modifies an organisation entry.
- `DELETE /admin/organisations/{org_id}` : Removes an organisation entry.
- `GET /admin/recent-activity` : Displays a timeline of recent critical system events.

Simulation Tools

- `POST /admin/simulator/generate` : Triggers the external language model service to construct a synthetic patient persona. The system registers a mock authentication user and backfills one month of synthetic health records based on the requested scenario.
- `POST /admin/patients/{patient_id}/generate-data` : Injects a month of synthetic health records into an existing patient account.

Chat History Module

The chat history module persists conversations between the patient and the conversational agent.

- `GET /chat/history` : Retrieves the chronological conversation log for the authenticated patient.
- `POST /chat/history` : Appends a new conversation entry to the log.
- `DELETE /chat/history` : Erases all conversation entries for the authenticated patient.

Deployment Details

The repository includes a configuration file explicitly formatted for the Vercel hosting environment. The configuration directs all incoming traffic to the primary FastAPI application script to handle routing natively.